

PROJECT REPORT
SMART CITY TRAFFIC MANAGEMENT SYSTEM

Using Graph Data Structure, Dijkstra Algorithm and BFS



L OVELY
P ROFESSIONAL
U NIVERSITY

Submitted by

Naureen, Shubham Yadav,

Saumya Mihir

12502577, 12500406, 12503940

Program:- Bachelor of Technology

Under the Guidance of

Mahipal Singh Papola, Deepak Kumar

Department of CSE

School of Computer Application

Lovely Professional University

ABSTRACT

The Smart City Traffic Management System is a web-based application developed to provide an efficient and interactive solution for route planning and traffic management within a city network. The primary objective of the system is to help users identify the shortest and most suitable route between a selected source and destination while providing additional information such as total distance, estimated travel time, traffic conditions, fuel cost, and nearby places.

The system represents cities and roads using the Graph Data Structure, where cities are treated as vertices or nodes and the roads connecting them are represented as weighted edges. The weight assigned to each edge represents the distance between two connected cities. The Dijkstra Shortest Path Algorithm is used to calculate the shortest route between the source and destination selected by the user. The project also incorporates the concept of Breadth-First Search (BFS) for graph traversal and exploration of connected nodes.

The application provides an interactive graphical representation of the city network using the HTML Canvas API. When a user searches for a route, the calculated shortest path is dynamically highlighted on the graph, making it easier to understand the working of the shortest-path algorithm. In addition, the system integrates an interactive map using Leaflet.js and OpenStreetMap, where city locations are represented using markers and the calculated route is visually highlighted on the map.

The frontend of the application is developed using HTML, CSS, and JavaScript, providing a modern, responsive, and user-friendly interface. The backend is implemented using Node.js and Express.js, which handles API requests and performs route-related calculations. The application also calculates estimated travel time according to the selected vehicle type and simulated traffic conditions. Additional features such as nearby place recommendations and estimated fuel costs enhance the usefulness of the system.

The frontend of the project is deployed using Netlify, while the backend API is hosted using Render. The complete source code is maintained using Git and GitHub for version control.

Overall, the Smart City Traffic Management System demonstrates the practical implementation of fundamental Data Structures and Algorithms, particularly Graphs, Dijkstra's Algorithm, and BFS, in solving real-world route optimization problems. The project combines algorithmic problem-solving with modern web development and interactive visualization techniques to provide an effective prototype for intelligent urban navigation and smart city applications.

1. INTRODUCTION

Rapid urbanization and continuous growth in the number of vehicles have made traffic management one of the major challenges faced by modern cities. Increasing traffic congestion can result in longer travel times, higher fuel consumption, increased transportation costs, and inconvenience for commuters. An efficient route planning system can help users select an appropriate path between different locations and improve the overall navigation experience.

The Smart City Traffic Management System is designed as a web-based route planning and traffic management application that demonstrates how Data Structures and Algorithms (DSA) can be applied to solve real-world transportation problems. The system allows users to select a source city, destination city, and vehicle type, after which it calculates and displays the shortest available route along with useful travel information.

The core structure of the system is based on the Graph Data Structure. In the implemented city network, each city is represented as a vertex or node, while a road connecting two cities is represented as an edge. Each road has an associated distance, which acts as the weight of the corresponding edge. This representation makes it possible to apply graph-based algorithms for route calculation and network traversal.

The Dijkstra Algorithm is the primary shortest-path algorithm used in the project. When the user selects a source and destination, the algorithm evaluates the available paths in the graph and determines the route with the minimum total distance. The project also includes the concept of Breadth-First Search (BFS) for graph traversal and exploring connected nodes in the city network.

Apart from calculating the shortest route, the application provides several additional features to improve the user experience. It displays the total route distance, estimated travel time, traffic condition, selected vehicle type, estimated fuel cost, and nearby places. The estimated travel time is calculated according to the distance, vehicle speed, and simulated traffic conditions.

To make the algorithm easier to understand visually, the project includes a City Network Graph created using the HTML Canvas API. The cities are displayed as nodes connected

through roads, and after a route is calculated, the selected shortest path is highlighted. This provides a visual demonstration of how graph-based route selection works.

The system also integrates an interactive digital map using Leaflet.js and OpenStreetMap. Markers are used to represent different cities, while the calculated route is highlighted on the map according to the sequence of cities returned by the shortest-path algorithm. This creates an interactive navigation experience and connects the algorithmic output with geographical visualization.

The application follows a client-server architecture. The frontend is developed using HTML, CSS, and JavaScript, whereas the backend is developed using Node.js and Express.js. The frontend communicates with the backend through REST-style API endpoints to retrieve city information, calculate routes, and display nearby places.

Overall, the Smart City Traffic Management System provides a practical demonstration of combining graph algorithms, backend development, frontend technologies, and interactive map visualization. It serves as a prototype showing how algorithmic concepts such as shortest-path calculation can be integrated into modern web applications for intelligent transportation and smart city solutions.

1.1 PROBLEM STATEMENT

Traffic congestion and inefficient route selection are common challenges in rapidly developing urban areas. As the number of vehicles and transportation requirements increase, commuters need an efficient mechanism to identify suitable routes between different locations. Selecting a route without considering distance or traffic conditions may result in increased travel time, unnecessary fuel consumption, and higher transportation costs.

Traditional route selection methods may require users to manually compare different roads and possible paths. In a city network containing multiple interconnected locations, finding the shortest route manually becomes increasingly difficult as the size of the network grows. Therefore, there is a need for an automated system that can represent the transportation network efficiently and calculate the shortest available route using appropriate algorithms.

The major problem addressed by the **Smart City Traffic Management System** is to determine an efficient route between a user-selected source and destination. The system must analyze multiple interconnected cities and roads and identify the path having the minimum total distance. To solve this problem, the project represents the transportation network using a

weighted graph, where cities are represented as nodes and roads are represented as weighted edges.

The **Dijkstra Shortest Path Algorithm** is used to determine the shortest route in the weighted city network. The system processes the source and destination selected by the user and returns the optimal path along with the total distance. This eliminates the need for manual route comparison and demonstrates how graph algorithms can be applied to transportation-related problems.

Another challenge addressed by the project is the effective visualization of algorithmic results. Displaying only the calculated distance may not provide users with a clear understanding of the selected route. Therefore, the application provides a graphical representation of the city network and highlights the calculated route. It also displays the route on an interactive map to provide better visual understanding.

The system further aims to provide useful travel-related information beyond basic shortest-path calculation. It estimates travel time according to the selected vehicle type and simulated traffic conditions, provides an estimated fuel cost, and displays nearby places associated with the selected city.

Therefore, the main problem can be summarized as:

To design and develop a web-based Smart City Traffic Management System that uses graph-based algorithms to calculate and visualize the shortest route between different cities while providing relevant travel information such as distance, estimated time, traffic condition, fuel cost, and nearby places.

The proposed system provides a simplified prototype of an intelligent urban navigation solution and demonstrates the practical application of **Data Structures and Algorithms** in solving real-world route optimization problems.

1.2 OBJECTIVES OF THE PROJECT

The primary objective of the Smart City Traffic Management System is to design and develop an interactive web-based application that demonstrates the practical implementation of Graph Data Structures and shortest-path algorithms for intelligent route planning. The system aims

to help users identify an efficient route between a selected source and destination while presenting relevant travel information in an easy-to-understand format.

The major objectives of the project are:

- To implement a Graph Data Structure:
The project aims to represent the city transportation network in the form of a weighted graph. Cities are represented as vertices or nodes, while roads connecting different cities are represented as edges. The distance between two connected cities is used as the weight of the corresponding edge.
- To calculate the shortest route using Dijkstra's Algorithm:
One of the main objectives is to implement Dijkstra's shortest-path algorithm to determine the minimum-distance route between the source and destination selected by the user.
- To demonstrate graph traversal using BFS:
The project incorporates the concept of Breadth-First Search (BFS) for systematic graph traversal and exploration of connected nodes, demonstrating another important graph algorithm.
- To provide interactive route visualization:
The application aims to visually represent the city network using the HTML Canvas API. When a route is calculated, the corresponding nodes and edges are highlighted to help users understand the selected path.
- To integrate an interactive map:
The system integrates Leaflet.js with OpenStreetMap to display geographical city locations. The calculated route is highlighted on the map to provide an interactive visualization of the source-to-destination path.
- To estimate travel time:
The application calculates approximate travel time based on the total route distance, selected vehicle type, assumed vehicle speed, and simulated traffic conditions. Travel durations shorter than one hour are displayed in minutes for better readability.
- To provide traffic information:
The project includes simulated traffic conditions such as Low, Medium, and High, which can be used to represent the effect of traffic on estimated travel time.
- To estimate fuel cost:
The system provides an approximate fuel cost based on the total distance of the

calculated route. This feature demonstrates how additional travel-related calculations can be integrated with route planning.

➤ To display nearby places:

The application provides information about predefined nearby places associated with the selected city, improving the usefulness of the navigation interface.

➤ To develop a responsive and user-friendly interface:

The project aims to provide a clean and modern user interface that allows users to easily select locations, search for routes, view results, and interact with graphical and map-based visualizations.

➤ To implement client-server architecture:

The system separates the frontend and backend components. The frontend handles user interaction and visualization, while the Node.js and Express.js backend processes API requests and route calculations.

➤ To deploy the complete application:

The final objective is to make the project accessible online by deploying the frontend and backend using cloud-based deployment platforms.

1.3 SCOPE OF THE PROJECT

The scope of the Smart City Traffic Management System focuses on developing a web-based prototype for intelligent route planning using graph-based data structures and algorithms. The project demonstrates how a transportation network can be represented digitally and how shortest-path algorithms can be used to assist users in selecting an efficient route between different locations.

The current implementation contains a network of selected cities, including Patna, Hajipur, Samastipur, Muzaffarpur, and Motihari. These cities are represented as nodes in a weighted graph, while the roads connecting them are represented as edges with associated distance values. The system can be extended in the future by adding more cities, roads, and dynamically updated traffic information.

The application allows users to select a source city, destination city, and vehicle type. Based on these inputs, the backend processes the city network and uses Dijkstra's Algorithm to calculate the shortest available route. The calculated path and total distance are then returned to the frontend and displayed to the user.

The scope of the system includes the following major functionalities:

- Selection of source and destination cities through an interactive route finder.
- Selection of different vehicle types such as Car, Bike, and Bus.
- Representation of cities and roads using a weighted Graph Data Structure.
- Calculation of the shortest route using Dijkstra's Algorithm.
- Implementation of BFS concepts for graph traversal and connected-node exploration.
- Visualization of the complete city network using the HTML Canvas API.
- Dynamic highlighting of the calculated shortest path on the city network graph.
- Integration of an interactive map using Leaflet.js and OpenStreetMap.
- Visualization of the calculated route on the map.
- Automatic adjustment of the map view according to the selected route.
- Display of route details, including source, destination, route sequence, and total distance.
- Display of simulated traffic conditions.
- Calculation of estimated travel time according to distance, vehicle speed, and traffic conditions.
- Calculation of an approximate fuel cost based on travel distance.
- Display of predefined nearby places for selected cities.
- Responsive and user-friendly web interface.
- REST-style communication between the frontend and Node.js/Express.js backend.
- Online deployment of frontend and backend components.

The current system is primarily designed as an educational and demonstration prototype. The traffic information used by the application is simulated rather than obtained from real-time traffic sensors or external traffic APIs. Similarly, the route highlighted on the interactive map connects the geographical coordinates of the cities in the calculated graph path and does not represent turn-by-turn road-level navigation.

In future implementations, the scope of the project can be expanded by integrating real-time traffic APIs, GPS-based current location detection, dynamic road conditions, accident alerts, road closures, public transportation information, emergency vehicle routing, and large-scale city networks.

Therefore, the present project establishes a foundation for understanding intelligent route planning systems and demonstrates how graph algorithms can be combined with modern web technologies to develop scalable smart city transportation solutions.

1.4 TECHNOLOGIES USED

The Smart City Traffic Management System is developed using a combination of frontend technologies, backend technologies, algorithms, visualization libraries, version control tools, and cloud deployment platforms. Each technology plays an important role in the overall functionality of the application.

1. HTML (HyperText Markup Language)

HTML is used to create the basic structure of the web application. It defines various components of the website, including the navigation bar, hero section, route finder, city network, live map, route summary, nearby places section, and footer.

The application uses HTML elements such as forms, selection menus, buttons, sections, and canvas elements to provide an organized user interface.

2. CSS (Cascading Style Sheets)

CSS is used to design and style the frontend of the application. It provides a modern and responsive appearance to the website. CSS is responsible for the layout, colors, typography, cards, buttons, traffic badges, shadows, spacing, hover effects, and responsive behavior of the system.

The user interface is designed to provide a simple and visually appealing experience across different screen sizes.

3. JavaScript

JavaScript is used to add dynamic functionality and interactivity to the frontend. It communicates with the backend APIs using the Fetch API and dynamically updates the webpage without requiring a complete page reload.

JavaScript is responsible for loading cities into dropdown menus, sending route requests, displaying route results, highlighting the shortest path on the graph, displaying nearby places, controlling the loading state of the route button, enabling smooth scrolling, and updating the route shown on the map.

4. Node.js

Node.js is used as the backend runtime environment of the application. It allows JavaScript to execute on the server side and handles the application's backend logic.

The Node.js backend receives requests from the frontend and processes route-related operations.

5. Express.js

Express.js is used as the backend web framework. It provides API endpoints through which the frontend communicates with the server.

The APIs are responsible for providing the list of cities, calculating routes, and returning nearby-place information.

6. Graph Data Structure

A weighted Graph Data Structure is used to represent the transportation network. Cities are represented as vertices, while roads are represented as weighted edges. The distance between two cities acts as the weight of an edge.

7. Dijkstra's Algorithm

Dijkstra's Algorithm is the primary shortest-path algorithm used in the system. It determines the minimum-distance path between the source and destination selected by the user.

8. Breadth-First Search (BFS)

BFS is included as a graph traversal technique for systematically exploring connected nodes in the network. It demonstrates another fundamental algorithm used for graph-based problems.

9. HTML Canvas API

The HTML Canvas API is used to create the visual representation of the city network. Cities are displayed as circular nodes and roads as connecting lines. The calculated shortest route is dynamically highlighted after a successful search.

10. Leaflet.js and OpenStreetMap

Leaflet.js is used to create the interactive map component of the application, while OpenStreetMap provides the map tiles. City markers are placed according to predefined geographical coordinates, and the calculated route is displayed as a highlighted polyline connecting the cities in the selected path.

11. Git and GitHub

Git is used for version control, allowing changes in the source code to be tracked and managed efficiently. GitHub is used to host the project's source-code repository and maintain the project files online.

12. Render

Render is used to deploy and host the Node.js and Express.js backend. The deployed backend provides the API services required by the frontend application.

13. Netlify

Netlify is used to deploy the frontend of the Smart City Traffic Management System, making the web application accessible through a public deployment link.

The combination of these technologies provides a complete development environment in which Data Structures and Algorithms, frontend development, backend APIs, interactive visualization, map integration, version control, and cloud deployment work together to create the Smart City Traffic Management System.

1.5 SYSTEM ARCHITECTURE AND WORKING

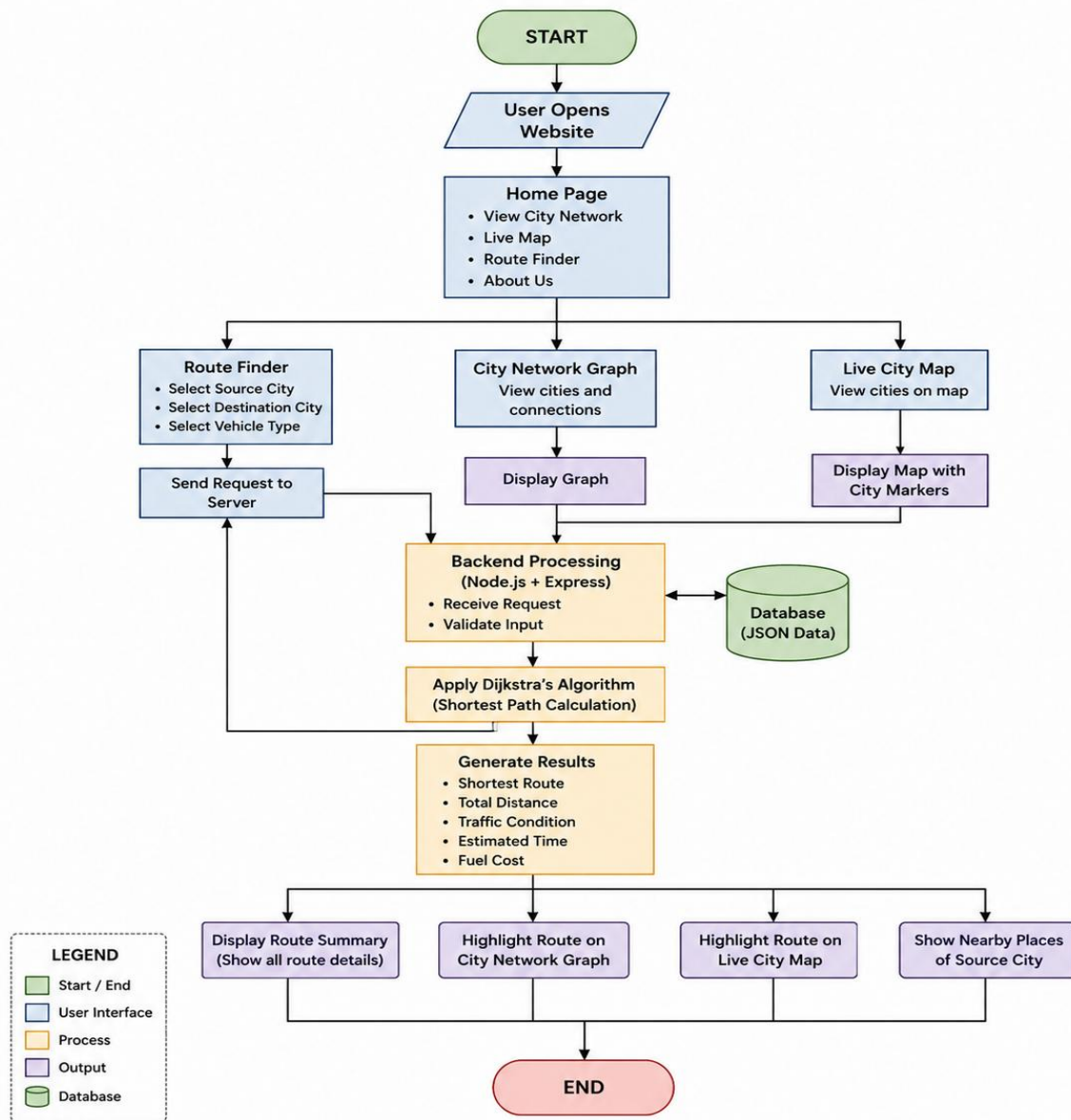
The Smart City Traffic Management System follows a client-server architecture, where the frontend is responsible for user interaction and visualization, while the backend handles route processing, graph algorithms, and travel-related calculations. The system is divided into multiple components so that each part performs a specific function.

System Architecture

The overall workflow of the system can be represented as follows:

1.5 SYSTEM FLOWCHART

The flowchart below shows the overall working of the Smart City Traffic Management System.



Working of the System

When the application is opened, the frontend sends a request to the backend to retrieve the available cities. These cities are dynamically displayed in the source and destination dropdown menus.

The user selects a source city, a destination city, and a vehicle type, such as Car, Bike, or Bus. After clicking the Find Best Route button, JavaScript sends an API request to the Node.js and Express.js backend containing the selected information.

The backend represents the available city network using a weighted graph. The Dijkstra Algorithm processes this graph and determines the shortest path from the selected source to the destination based on road distances.

After the shortest route is identified, the route-processing service calculates additional information. This includes the estimated travel time according to the selected vehicle speed and simulated traffic condition. An approximate fuel cost is also calculated using the total route distance.

The backend then sends the calculated information to the frontend as a response. The frontend dynamically displays the complete route summary, including the route sequence, total distance, vehicle type, traffic condition, estimated travel time, and fuel cost.

The calculated route is also passed to the Canvas-based City Network, where the nodes and edges belonging to the selected route are highlighted. At the same time, the route is drawn on the Leaflet interactive map by connecting the geographical coordinates of the cities included in the calculated path.

The application additionally requests nearby-place information for the selected source city and displays the corresponding locations in the Nearby Places section.

This architecture separates the user interface, algorithmic processing, business logic, and visualization components, making the application easier to understand, maintain, and extend. It also demonstrates how a graph-based shortest-path algorithm can be integrated into a full-stack web application to provide an interactive smart city navigation solution.

2. ALGORITHM

Algorithms are the core components of the Smart City Traffic Management System. The application uses graph-based concepts to represent the city road network and determine an efficient route between a selected source and destination.

The major algorithms and data structures used in this project are Graph Data Structure, Dijkstra's Shortest Path Algorithm, and Breadth-First Search (BFS).

2.1 GRAPH DATA STRUCTURE

A Graph is a non-linear data structure consisting of a collection of vertices (nodes) and edges. Graphs are commonly used to represent networks such as road systems, computer networks, social networks, and transportation systems.

In this project, the city transportation network is represented as a weighted graph.

- Vertices (Nodes): Represent cities.
- Edges: Represent roads connecting two cities.
- Weights: Represent the distance between two connected cities.

The graph used in the project contains the following cities:

Patna

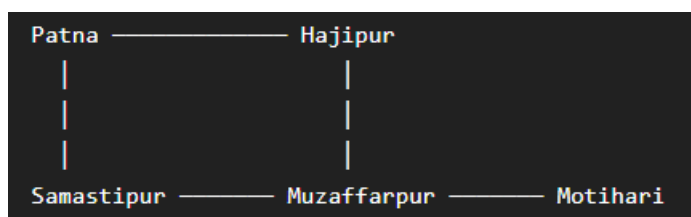
Hajipur

Samastipur

Muzaffarpur

Motihari

The road network is represented as:



The weighted road information used by the system includes:

Source	Destination	Distance	Traffic
Patna	Hajipur	20 KM	Low
Hajipur	Muzaffarpur	52 KM	High
Patna	Samastipur	85 KM	Medium
Samastipur	Muzaffarpur	40 KM	Low
Muzaffarpur	Motihari	60 KM	Medium

For example, if a user wants to travel from Patna to Motihari, multiple paths may be available in the graph.

Route 1:

```
Patna → Hajipur → Muzaffarpur → Motihari
```

Total Distance:

```
20 + 52 + 60 = 132 KM
```

Route 2:

```
Patna → Samastipur → Muzaffarpur → Motihari
```

Total Distance:

```
85 + 40 + 60 = 185 KM
```

Since the first route has a shorter total distance, the shortest-path algorithm selects:

```
Patna → Hajipur → Muzaffarpur → Motihari
```

with a total distance of 132 KM.

The Graph Data Structure therefore acts as the foundation of the Smart City Traffic Management System. Once the city network is represented as a graph, algorithms such as Dijkstra's Algorithm can efficiently process the network and determine the shortest route between different locations.

2.2 DIJKSTRA'S SHORTEST PATH ALGORITHM

Dijkstra's Algorithm is used in the Smart City Traffic Management System to find the shortest route between two cities. It works on a weighted graph, where cities are represented as nodes and roads are represented as edges. The distance between two cities is considered the weight of an edge.

When the user selects a source city and a destination city, Dijkstra's Algorithm checks the available routes and selects the route with the minimum total distance.

How It Works

The working of Dijkstra's Algorithm can be explained in the following simple steps:

1. Start from the source city.
2. Set the distance of the source city to 0.
3. Check all the neighboring cities connected to the current city.
4. Calculate the distance to each neighboring city.
5. Select the city having the shortest distance.
6. Continue the same process until the destination is reached.
7. Return the shortest route and its total distance.

Example

Suppose the user wants to travel from Patna to Motihari.

There are two possible routes:

```
Route 1:  
Patna → Hajipur → Muzaffarpur → Motihari  
  
Distance = 20 + 52 + 60  
          = 132 KM
```

```
Route 2:  
Patna → Samastipur → Muzaffarpur → Motihari  
  
Distance = 85 + 40 + 60  
          = 185 KM
```

Since 132 KM is less than 185 KM, Dijkstra's Algorithm selects:

```
Patna → Hajipur → Muzaffarpur → Motihari
```

Therefore, the shortest distance is **132 KM**.

Use in the Project

The basic working of the algorithm in the project is:

```
User Selects Source and Destination  
↓  
Clicks "Find Best Route"  
↓  
Dijkstra Algorithm Runs  
↓  
Finds Shortest Route  
↓  
Calculates Total Distance  
↓  
Displays Result to the User
```

Thus, Dijkstra's Algorithm helps the Smart City Traffic Management System find the shortest available route between the selected source and destination.

2.3 BREADTH-FIRST SEARCH (BFS)

Breadth-First Search (BFS) is a graph traversal algorithm used to visit and explore the nodes of a graph. It visits the nearest connected nodes first and then moves to the next level.

In the Smart City Traffic Management System, BFS is included to demonstrate graph traversal and to explore connected cities in the city network.

How BFS Works

The working of BFS can be explained in simple steps:

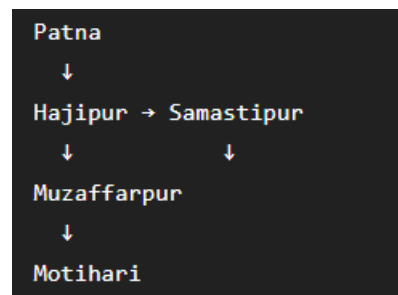
1. Start from a selected city.
2. Mark the city as visited.

3. Add the city to a queue.
4. Visit all the neighboring cities.
5. Add unvisited neighboring cities to the queue.
6. Repeat the process until all connected cities are visited.

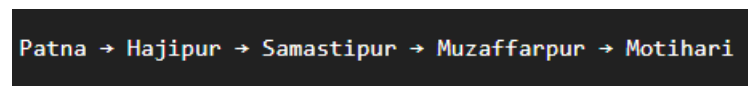
Example

Suppose BFS starts from Patna.

The city network may be visited as:

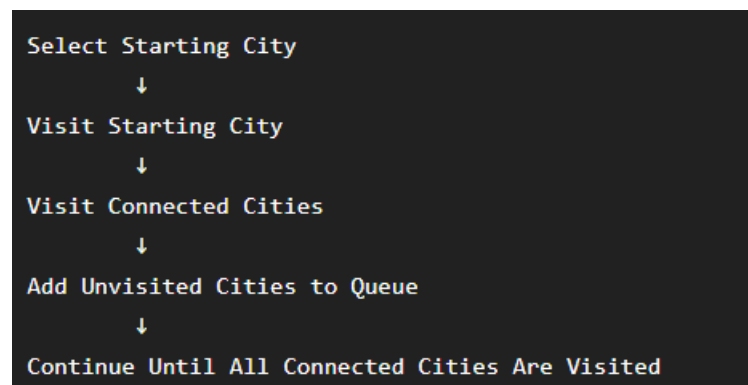


A possible BFS traversal is:



BFS in the Project

The basic flow is:



In this project, Dijkstra's Algorithm is mainly used for finding the shortest route based on distance, while BFS is used as a graph traversal concept for exploring connected cities.

Therefore, BFS demonstrates how the system can systematically visit different cities present in the graph network.

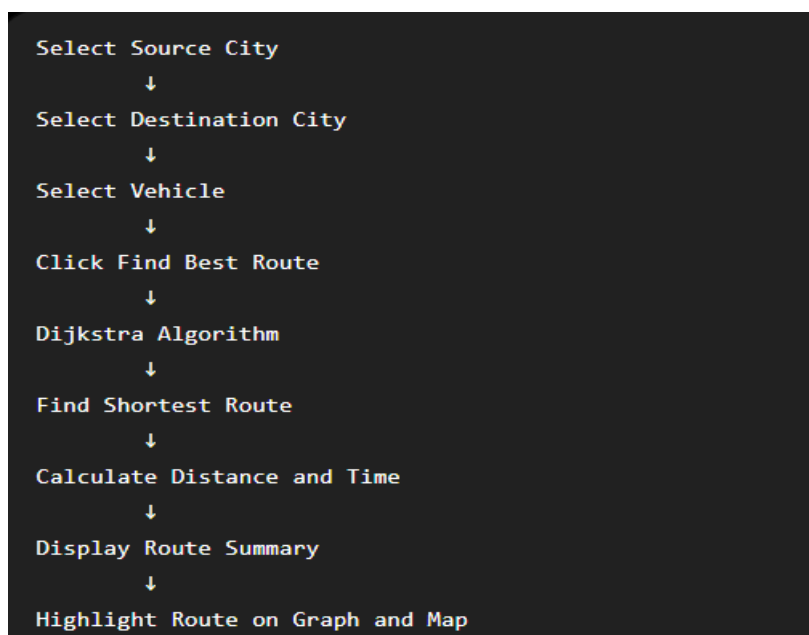
2.4 ROUTE CALCULATION PROCESS

The **Smart City Traffic Management System** calculates the best route using the city network and Dijkstra's Algorithm.

When the user selects a source city, destination city, and vehicle type, the system follows these simple steps:

1. The user selects the **Source City**.
2. The user selects the **Destination City**.
3. The user selects a vehicle such as **Car, Bike, or Bus**.
4. The user clicks the **Find Best Route** button.
5. The request is sent to the backend server.
6. Dijkstra's Algorithm finds the shortest route.
7. The system calculates the total distance.
8. Estimated travel time and fuel cost are calculated.
9. The final route details are displayed to the user.
10. The selected route is highlighted on the **City Network Graph** and the **Live Map**.

Simple Workflow:-



For example, if the user selects **Patna** as the source and **Motihari** as the destination, the system finds the shortest available path and displays the complete route with its distance, estimated travel time, traffic condition, and fuel cost.

This process makes the application simple and easy to use while demonstrating the practical use of shortest-path algorithms in route planning.

3. RESULTS

The **Smart City Traffic Management System** was successfully developed and deployed as a web-based application. The final system provides an interactive and user-friendly interface through which users can find the shortest route between different cities and view important travel-related information.

The application allows the user to select a **source city, destination city, and vehicle type**, such as Car, Bike, or Bus. After selecting the required information, the user clicks the **Find Best Route** button. The system then processes the request and uses **Dijkstra's Algorithm** to calculate the shortest available route between the selected cities.

The calculated result is displayed in the **Route Summary**, which includes the source city, destination city, shortest route, total distance, selected vehicle, traffic condition, estimated travel time, and estimated fuel cost. This provides all important route information in a simple and easy-to-understand format.

The application also provides a visual representation of the city network using the **HTML Canvas API**. Cities are displayed as nodes and roads are represented as connecting lines. After the shortest route is calculated, the selected path is highlighted on the City Network Graph, allowing the user to visually identify the route selected by the algorithm.

An interactive **Live City Map** is also integrated using **Leaflet.js and OpenStreetMap**. The cities are displayed using markers on the map. When a route is calculated, the path between the selected cities is highlighted on the map, providing a geographical representation of the calculated route.

The **Nearby Places** feature displays useful predefined locations associated with the selected city. This provides additional location-based information along with the route details.

The system was tested using different combinations of source cities, destination cities, and vehicle types. During testing, the application successfully loaded the available cities, calculated the shortest route, displayed the total distance, estimated the travel time and fuel cost, and showed the traffic condition. The calculated route was successfully highlighted on both the City Network Graph and the Live City Map. The navigation links and buttons were also tested and worked correctly.

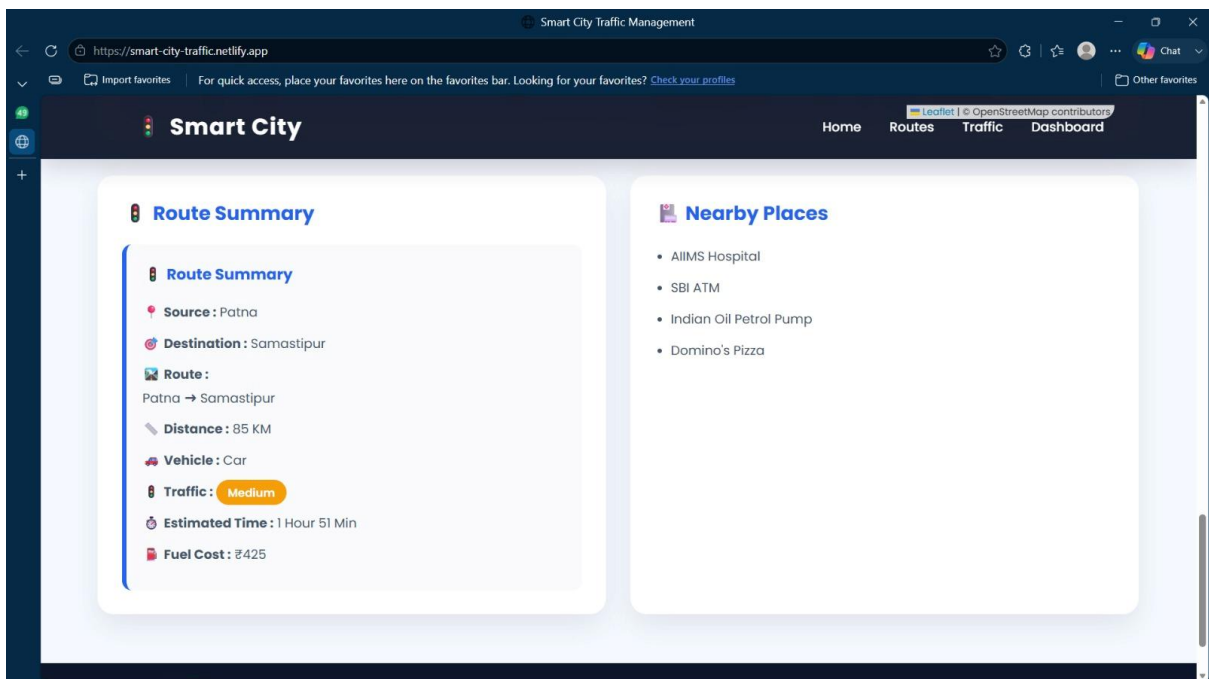
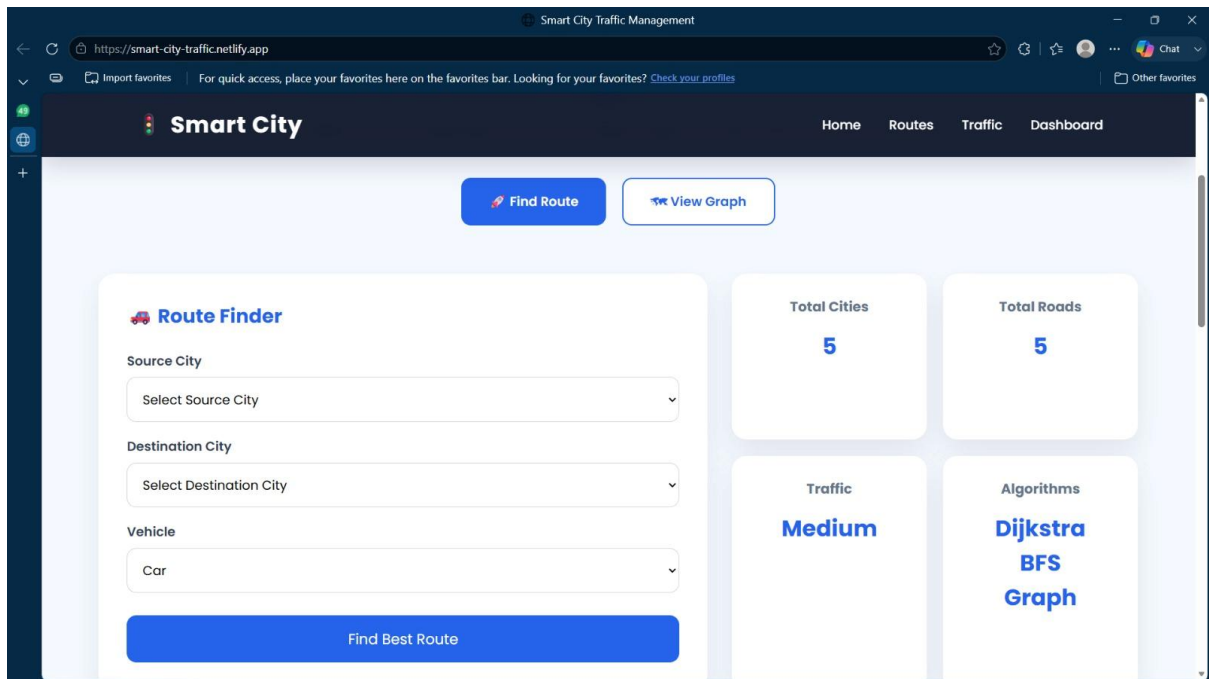
The frontend and backend communication was successfully implemented using API requests. The frontend of the application was deployed on **Netlify**, while the Node.js and Express.js

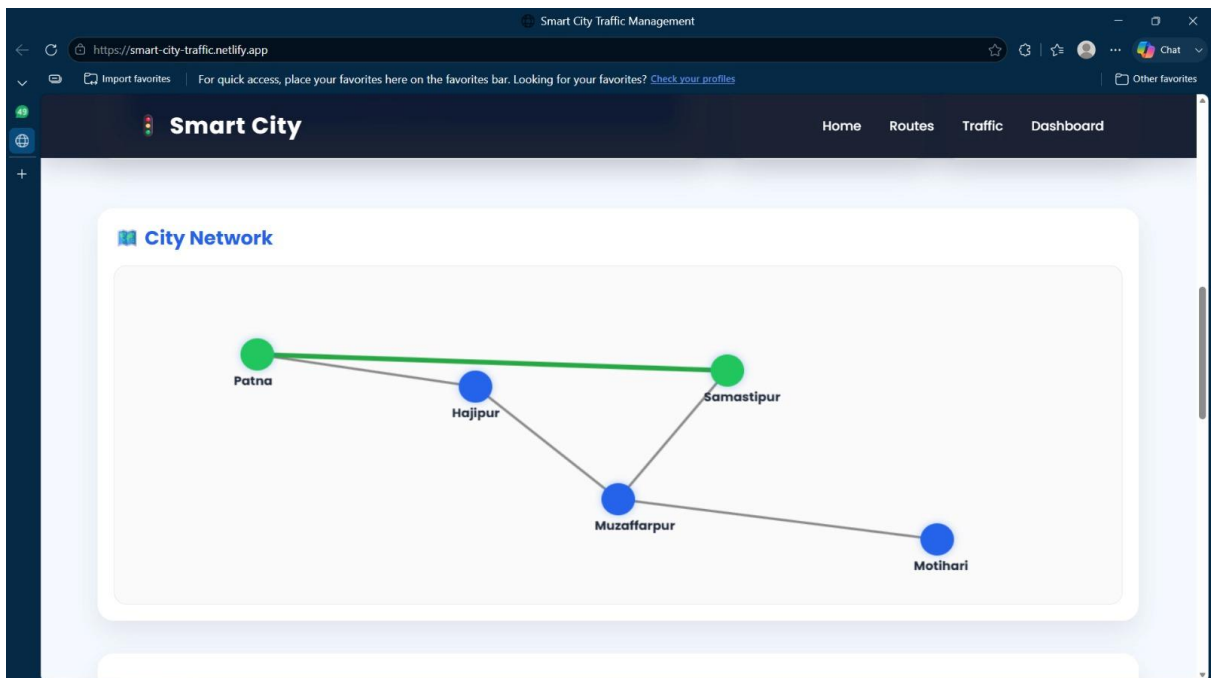
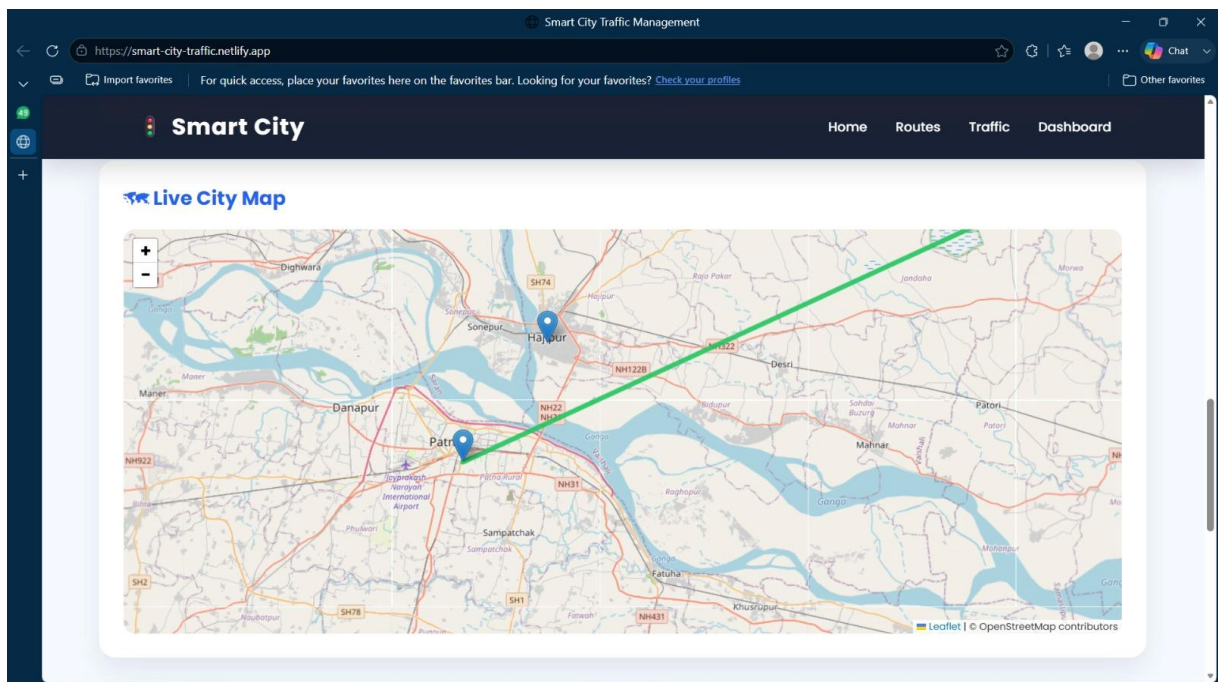
backend was deployed on **Render**. The deployed application successfully communicates with the backend APIs to retrieve city information, calculate routes, and display nearby places.

Overall, the results show that the Smart City Traffic Management System successfully demonstrates the practical implementation of **Graph Data Structure, Dijkstra's Algorithm, BFS, web development, and route visualization**. The system provides a simple prototype of a smart navigation solution that can calculate and visually display the shortest route between different cities.

Project Output







4. CONCLUSION

The **Smart City Traffic Management System** was successfully developed as a web-based application for finding the shortest route between different cities. The project demonstrates the practical use of **Graph Data Structure, Dijkstra's Algorithm, and BFS** in solving route-planning problems.

The system allows users to select a source city, destination city, and vehicle type. It calculates the shortest route and displays useful information such as total distance, traffic condition, estimated travel time, and fuel cost. The application also provides a **City Network Graph, interactive Live Map, route highlighting, and nearby places**, making the system easy to understand and use.

The frontend was developed using **HTML, CSS, and JavaScript**, while the backend was developed using **Node.js and Express.js**. Leaflet.js and OpenStreetMap were used for map visualization. The project was successfully deployed online using Netlify for the frontend and Render for the backend.

Overall, this project helped in understanding how **Data Structures and Algorithms can be integrated with web development** to create a practical application. In the future, the system can be improved by adding real-time traffic information, GPS location tracking, more cities and roads, accident alerts, and dynamic route updates.

5. REFERENCES

The following resources were referred to during the development of the **Smart City Traffic Management System**:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press.
2. GeeksforGeeks. **Dijkstra's Shortest Path Algorithm**.
3. GeeksforGeeks. **Breadth-First Search (BFS) for a Graph**.
4. GeeksforGeeks. **Graph Data Structure and Algorithms**.
5. MDN Web Docs. **HTML, CSS, and JavaScript Documentation**.
6. Node.js. **Official Node.js Documentation**.
7. Express.js. **Official Express.js Documentation**.
8. Leaflet.js. **Official Leaflet JavaScript Library Documentation**.
9. OpenStreetMap. **Open-Source Map Data and Map Services**.
10. GitHub. **Git Repository Hosting and Version Control Platform**.
11. Netlify. **Frontend Web Application Deployment Platform**.
12. Render. **Cloud Platform for Backend Application Deployment**.

PROJECT LINKS

Source Code:

<https://github.com/saumyamihir/Smart-City-Traffic>

Deployment Link:

<https://smart-city-traffic.netlify.app/>